

# Rapport Planning Poker

## 1. Introduction et contexte

Ce projet a pour objectif de réaliser une application web de Planning Poker permettant à une équipe agile d'estimer l'effort des tâches d'un backlog de développement logiciel de manière collaborative et structurée. L'application offre la possibilité de créer des sessions de vote, d'y faire participer plusieurs joueurs en temps réel, de recueillir leurs estimations sous forme de cartes, puis de calculer une valeur d'estimation finale pour chaque tâche.

Le travail a été mené dans le cadre du module de Conception Agile et a été réalisé en binôme (Mongi et Tom), en s'appuyant sur les principes du pair programming. Concrètement, le développement s'est organisé en itérations courtes sur des fonctionnalités ciblées (gestion des sessions, interface de vote, sauvegarde/restauration, etc.), avec alternance régulière des rôles de *driver* (écriture du code) et de *navigator* (relecture, conception, identification des cas limites), ce qui a permis d'améliorer la qualité du code et de limiter les régressions.

---

## 2. Choix techniques et architecture

### 2.1 Langage et frameworks

Le projet s'appuie sur une architecture full JavaScript :

- Backend :
  - Node.js avec Express pour fournir un serveur HTTP simple, servant les fichiers statiques et exposant une API REST minimale pour la gestion des sessions (`/api/create-session`, `/api/session/:code`).
  - Socket.io côté serveur pour la communication temps réel avec les clients (connexion des joueurs, réception des votes, diffusion des mises à jour d'état).
  - cors pour autoriser les requêtes cross-origin en environnement de développement.
- Frontend :
  - JavaScript vanilla (ES6+) sans framework SPA (type React/Vue), afin de garder une base de code légère, lisible et facile à appréhender dans un contexte pédagogique.
  - Utilisation des modules ES (`import` / `export`) pour structurer la logique en fichiers indépendants : `core/PlanningPokerClient.js`, `ui/components/*.js`, `ui/screens/*.js`.

Ces choix sont adaptés au besoin pour plusieurs raisons :

- Le Planning Poker est une activité fortement interactive qui nécessite un retour en temps réel (affichage des joueurs connectés, de leurs statuts de vote, des résultats dès que tous ont voté), ce que Socket.io gère très bien côté web.
- L'utilisation de JavaScript à la fois côté client et côté serveur simplifie le projet, évite le changement de paradigme entre deux langages et réduit la courbe d'apprentissage.
- Express et Socket.io offrent une solution éprouvée et peu verbeuse pour monter rapidement un serveur web temps réel et se concentrer sur la logique métier plutôt que sur l'infrastructure.

## 2.2 Architecture logicielle

L'architecture du code distingue clairement plusieurs couches :

- Couche cœur (core) côté client :
  - `core/PlanningPokerClient.js` définit la classe `PlanningPokerClient` qui joue le rôle de contrôleur principal côté client.
  - Cette classe gère :
    - la connexion Socket.io (`initializeSocket`, `setupSocketHandlers`) ;
    - l'état de la session (code de session, pseudo du joueur, mode de jeu, créateur) ;
    - l'état local du jeu (`gameState`) contenant les joueurs, le backlog, l'index de tâche courante, les votes, les tâches complétées et les estimations finales ;
    - la navigation entre les différents écrans de l'interface (menu, configuration, jeu, résultats, reprise, etc.).
- Couche UI (présentation) :
  - `ui/screens/*.js` regroupe les modules responsables des écrans complets :
    - `menuScreen.js` pour le menu principal ;
    - `setupScreen.js` pour l'écran de configuration et d'attente des joueurs ;
    - `gameScreen.js` pour l'écran de vote ;
    - `resultsScreen.js`, `taskCompletedScreen.js`, `coffeeBreakScreen.js`, `gameOverScreen.js`, `resumeWaitingScreen.js`, `resumeSummaryScreen.js`, `addTasksScreen.js` pour les étapes spécifiques du flux de jeu.
  - `ui/components/*.js` contient des composants réutilisables :
    - `CardGrid.js` pour la grille de cartes (1, 2, 3, 5, 8, 13, 20, 40, 100, ?,  ) ;
    - `PlayerList.js` pour la liste des joueurs et leur statut de vote ;
    - `ProgressBar.js` pour la barre de progression des votes ;
    - `ConnectionStatus.js` pour l'indicateur visuel de connexion/déconnexion ;
    - `WaitingPlayers.js` pour l'affichage des joueurs déjà connectés et prêts.
- Backend (serveur) :

- `server.js` instancie Express, configure Socket.io, maintient l'état des sessions et implémente les événements principaux : `join-session`, `cast-vote`, `load-backlog`, `continue-new-round`, `continue-next-round`, `leave-session`.
- `socketHandlers.js` factorise la logique de gestion des événements et manipule `gameState` côté serveur.

Ce découpage correspond à une architecture de type MVC allégée :

- Model : l'état du jeu (`gameState`, backlog, votes) coté serveur et synchronisé vers les clients.
- View : les modules d'interface qui manipulent le DOM (`ui/screens` et `ui/components`).
- Controller : `PlanningPokerClient` côté client et les handlers Socket.io côté serveur.

Cette organisation facilite la maintenance, la réutilisation de composants, ainsi que l'écriture de tests unitaires sur la logique métier indépendamment de l'interface.

## 2.3 Modélisation

Plusieurs classes conceptuelles structurent le modèle de données :

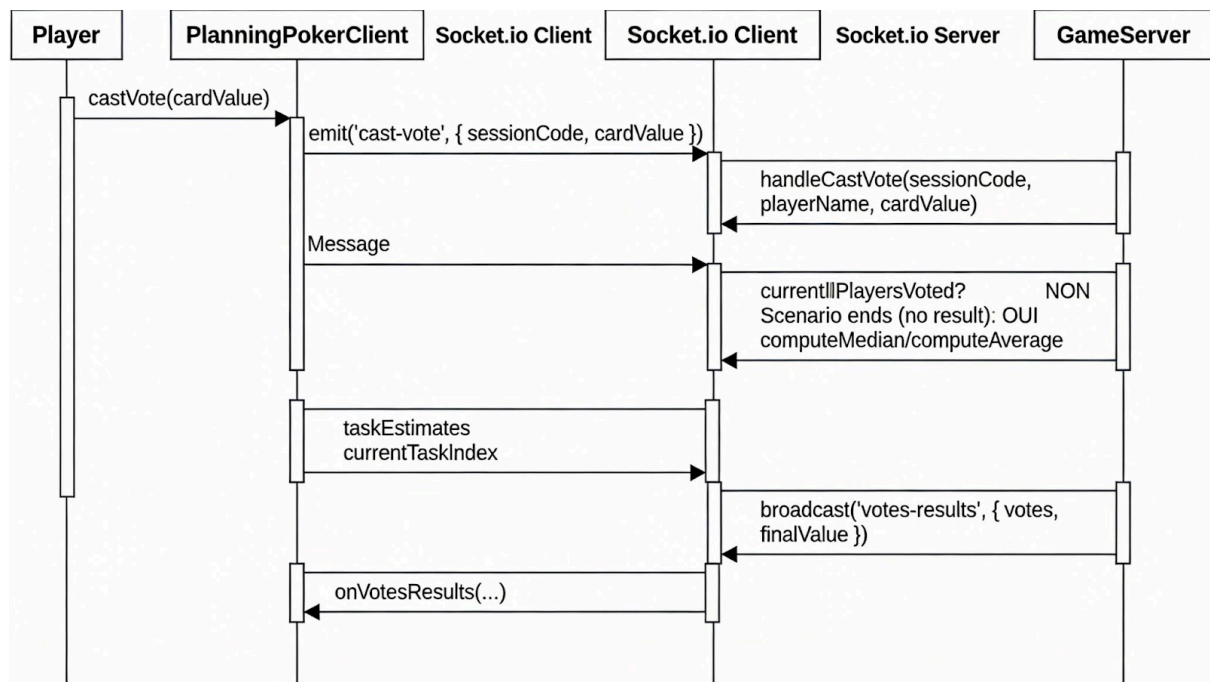
| Player                              | Task                                                       | GameState                                                                                         |
|-------------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| name : string<br>hasVoted : boolean | id : number<br>name : string                               | players : List<Player><br>backlog : List<Task>                                                    |
|                                     | description : string<br><br>finalValue : number (nullable) | currentTaskIndex : number<br>currentTaskVotes : Map<Player, number>                               |
|                                     | completed : boolean<br>completedAt : Date (nullable)       | completedTasks : List<number><br>taskEstimates : Map<number, number><br>currentTaskRound : number |

- Joueur (Player)  
Un joueur est identifié par :
  - `name` : pseudo unique dans une session ;
  - `hasVoted` : booléen indiquant s'il a voté pour la tâche courante ;
  - éventuellement un vote courant enregistré côté serveur dans `currentTaskVotes[playerName]`.
- Tâche (Task)  
Une tâche du backlog contient :
  - `id` : identifiant numérique ;
  - `name` : nom de la tâche ;

- `description` : description fonctionnelle ;
- `finalValue` : estimation finale choisie ;
- `completed` : booléen indiquant si la tâche a été complètement estimée ;
- `completedAt` : horodatage de la complétion.
- État du jeu (GameState)  
Le module `gameState.js` définit un `defaultGameState` qui contient notamment :
  - `players` : liste des joueurs ;
  - `backlog` : tableau de tâches ;
  - `currentTaskIndex` : index de la tâche en cours ;
  - `currentTaskVotes` : votes courants pour la tâche ;
  - `completedTasks` : liste des identifiants de tâches complétées ;
  - `taskEstimates` : `map taskId -> valeur finale` ;
  - `currentTaskRound` : numéro de round pour la tâche en cours.

Les diagrammes de classes insérés dans le rapport peuvent se concentrer sur ces entités et leurs relations, sans chercher à représenter tous les fichiers techniques, afin de rester lisibles.

Un diagramme de séquence pertinent est celui d'un tour de vote :



- Le joueur appelle `castVote(cardValue)` côté client ;
- Le client émet `cast-vote` via Socket.io ;
- Le serveur met à jour `currentTaskVotes`, teste si tous les joueurs ont voté et, le cas échéant, calcule la médiane ou la moyenne, met à jour `taskEstimates` et notifie tous les clients avec les résultats.

## 2.4 Gestion des données JSON

Le projet manipule deux familles de fichiers JSON :

1. Backlog minimal pour démarrer une partie :
  - Une simple liste de tâches avec `id`, `name`, `description`, chargée par `loadBacklogFile` ou collée dans une zone de texte sur l'écran d'ajout de tâches.
2. Sauvegarde complète de partie générée par `downloadBacklog()` :
  - metadata :
    - `sessionCode`, `participantCount`, `participants`, `gameMode` ;
    - `completedTasks`, `taskEstimates`, `currentTaskIndex`, `currentTaskVotes`, `currentTaskRound` ;
  - tasks : backlog enrichi avec `finalValue`, `completed`, `completedAt` pour les tâches terminées ;
  - votes : historique des votes par tâche (`taskId -> { playerName -> value }`).

Ces structures permettent :

- de rejouer une partie avec exactement les mêmes participants et le même backlog ;
  - de conserver un historique des estimations pour des audits ou des rétrospectives.
- 

## 3. Mise en place de l'intégration continue

### 3.1 Workflow d'intégration continue

L'intégration continue est basée sur Node.js et npm.

Un pipeline de CI a été configuré pour exécuter, à chaque push :

1. Checkout du dépôt ;
2. Installation des dépendances : `npm install` ;
3. Exécution des tests unitaires : `npm test` ;
4. Génération de la documentation via JSDoc : `npm run docs`.

Ce workflow garantit qu'aucune modification ne casse la logique métier ou les règles de vote sans être détectée.

## 3.2 Tests unitaires automatiques

Le projet utilise Jest comme framework de tests unitaires, déclaré en devDependency avec une configuration spécifique à la racine du `package.json` :

```
1  name: Tests unitaire Javascript
2  on:
3    push:
4      branches:
5        - main
6    pull_request:
7      branches:
8        - main
9
10 jobs:
11   test:
12     runs-on: ubuntu-latest
13
14     steps:
15       - name: Checkout code
16         uses: actions/checkout@v4
17
18       - name: Set up node
19         uses: actions/setup-node@v4
20
21       - name: Install dependencies
22         run: npm install
23       - name: Run tests
24         run: npm run test
```

```
1  ► Run npm run test
4
5  > planning-poker-pro@1.0.0 test
6  > jest
7
8  console.log
9  ☕ Pause café - Session ABC123 FERMÉE
10
11    at log (server.js:78:17)
12
13  console.log
14  📋 Participants sauvegardés: Alice, Bob
15
16    at log (server.js:79:17)
17
18  PASS ./server.test.js
19    ✓ calculateMedian ignore coffee et ? et calcule la médiane (2 ms)
20    ✓ isUnanimous retourne true quand tous les votes sont identiques
21    ✓ isUnanimous retourne false quand les votes sont différents (1 ms)
22    ✓ checkCoffeeBreak ferme la session quand tous les joueurs votent coffee (20 ms)
23    ✓ checkVotingCondition en mode strict sans unanimité demande un nouveau round (1 ms)
24
25  Test Suites: 1 passed, 1 total
26  Tests:      5 passed, 5 total
27  Snapshots:  0 total
28  Time:       0.57 s
29  Ran all test suites.
```

Les tests sont pensés pour couvrir principalement :

- la logique de calcul de la valeur finale d'une tâche (moyenne, médiane) en fonction des votes reçus ;
- la vérification des conditions d'unanimité lors du premier tour de vote ;
- la validation de la structure des JSON de backlog et de sauvegarde.

Dans la CI, l'exécution de `npm test` et `npm run test:coverage` permet d'imposer un seuil minimal de couverture globale, ce qui incite à tester systématiquement les évolutions.

### 3.3 Génération de documentation automatique

L'ensemble des fichiers principaux du projet est documenté avec JSDoc (en-têtes de fichiers, description des fonctions, types de paramètres et valeurs de retour). C'est le cas notamment de :

- `PlanningPokerClient.js` (classe principale, méthodes de navigation, de rendu et de logique métier) ;
- `CardGrid.js`, `PlayerList.js`, `ProgressBar.js`, `ConnectionStatus.js`, `WaitingPlayers.js` (composants UI).

Dans un pipeline CI complet, un job a été implémenté:

---

```
1  name: Generate JSDoc Documentation
2
3  on:
4    push:
5      branches:
6        - main
7  permissions:
8    contents: write
9  jobs:
10   build:
11     runs-on: ubuntu-latest
12
13     steps:
14       - name: Checkout repository
15         uses: actions/checkout@v4
16
17       - name: Setup Node
18         uses: actions/setup-node@v4
19
20       - name : Install JSDoc
21         run : npm install -g jsdoc
22
23       - name: Generate Documentation
24         run: npm run docs
25
26       - name: Deploy Documentation
27
28         uses: peaceiris/actions-gh-pages@v3
29         with:
30           github_token: ${ secrets.GITHUB_TOKEN }
31           publish_dir: ./docs
```

```
39  ✓ Dependances installées
40
41  ✨ Création de la configuration JSDoc...
42
43  📁 Chemins détectés:
44  ✓ /home/runner/work/Projet_Planning_Poker/Projet_Planning_Poker/public/src/core
45  ✓ /home/runner/work/Projet_Planning_Poker/Projet_Planning_Poker/public/src/ui/screens
46  ✓ /home/runner/work/Projet_Planning_Poker/Projet_Planning_Poker/public/src/ui/components
47  ✓ /home/runner/work/Projet_Planning_Poker/Projet_Planning_Poker/public/src/utls
48  ✓ /home/runner/work/Projet_Planning_Poker/Projet_Planning_Poker/server.js
49  ✓ README.md trouvé et ajouté à la configuration
50  ✓ Configuration créée: /home/runner/work/Projet_Planning_Poker/Projet_Planning_Poker/jsdoc.json
51
52  🚀 Génération de la documentation JSDoc...
53  Exécution: npx jsdoc -c "/home/runner/work/Projet_Planning_Poker/Projet_Planning_Poker/jsdoc.json"
54  ✓ Documentation générée avec succès
55  ✓ Fichier index.md créé
56
```

Cela permet de disposer d'une documentation HTML consultable sans ouvrir le code source.

---

## 4. Manuel utilisateur et fonctionnalités

### 4.1 Installation et lancement

Pour exécuter le projet sur la machine de l'évaluateur :

1. Cloner le dépôt :

bash

```
git clone <URL_DU_DEPOT>
```

```
cd planning-poker
```

2. Installer les dépendances :

bash

```
npm install
```

3. Démarrer le serveur :

bash

```
npm start
```

4. Ouvrir l'application dans un navigateur :

text

```
http://localhost:3000
```



L'application est alors prête à être utilisée pour créer ou rejoindre une session de Planning Poker.

## 4.2 Modes de jeu et règles particulières

L'application supporte notamment :

- Mode Strict :
  - La révélation des votes n'a lieu que lorsque tous les joueurs ont voté (`hasVoted = true` pour chacun).
  - Avant cela, seuls les statuts de vote (« En attente » / « Voté ✓ ») sont visibles, pas les valeurs.
- Mode basé sur une agrégation (médiane / moyenne) :
  - Une fois les votes révélés, une valeur finale (médiane ou moyenne selon la règle choisie) est calculée côté serveur à partir de l'ensemble des votes pour la tâche.
  - Cette valeur est stockée dans `taskEstimates` et associée à la tâche dans le fichier de sauvegarde.
- Premier tour à l'unanimité :
  - Pour le premier round sur une tâche, si tous les joueurs votent la même valeur, la tâche peut être immédiatement considérée comme estimée (unanimité).
  - Dans le cas contraire, un second round peut être lancé (`startNewRound`) après discussion.

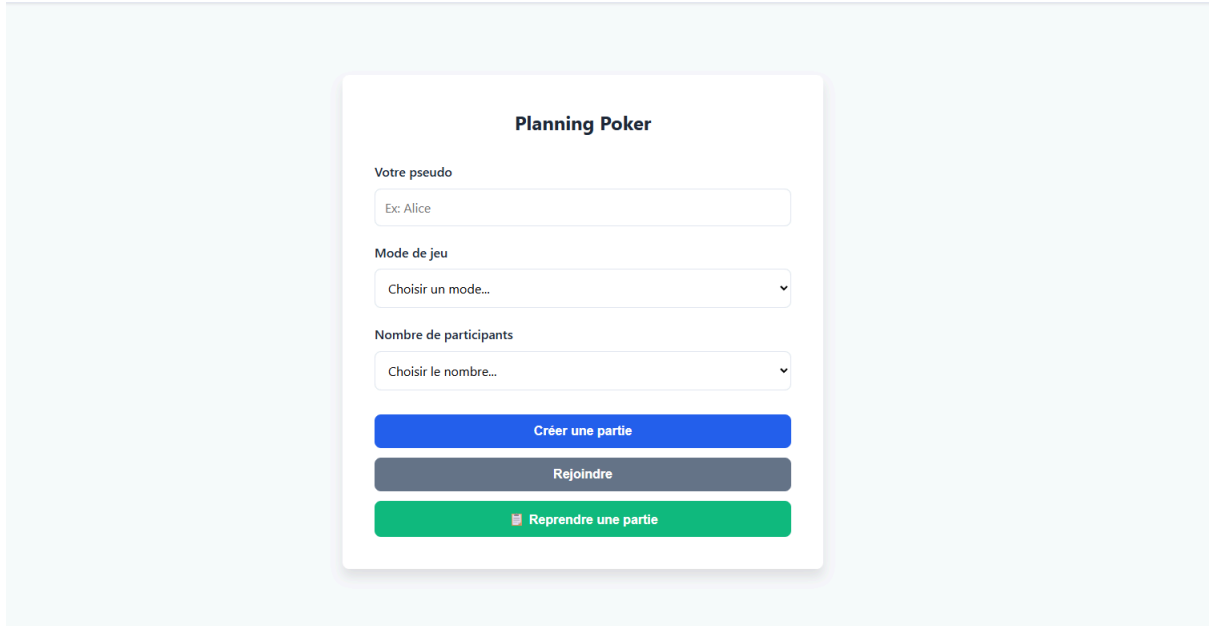
Ces règles respectent l'esprit du sujet, qui demande notamment de gérer un mode strict et de traiter le cas particulier d'un premier tour à l'unanimité.

## 4.3 Interface et déroulement d'une partie

L'interface utilisateur est structurée en plusieurs écrans :

 **Planning Poker Pro**

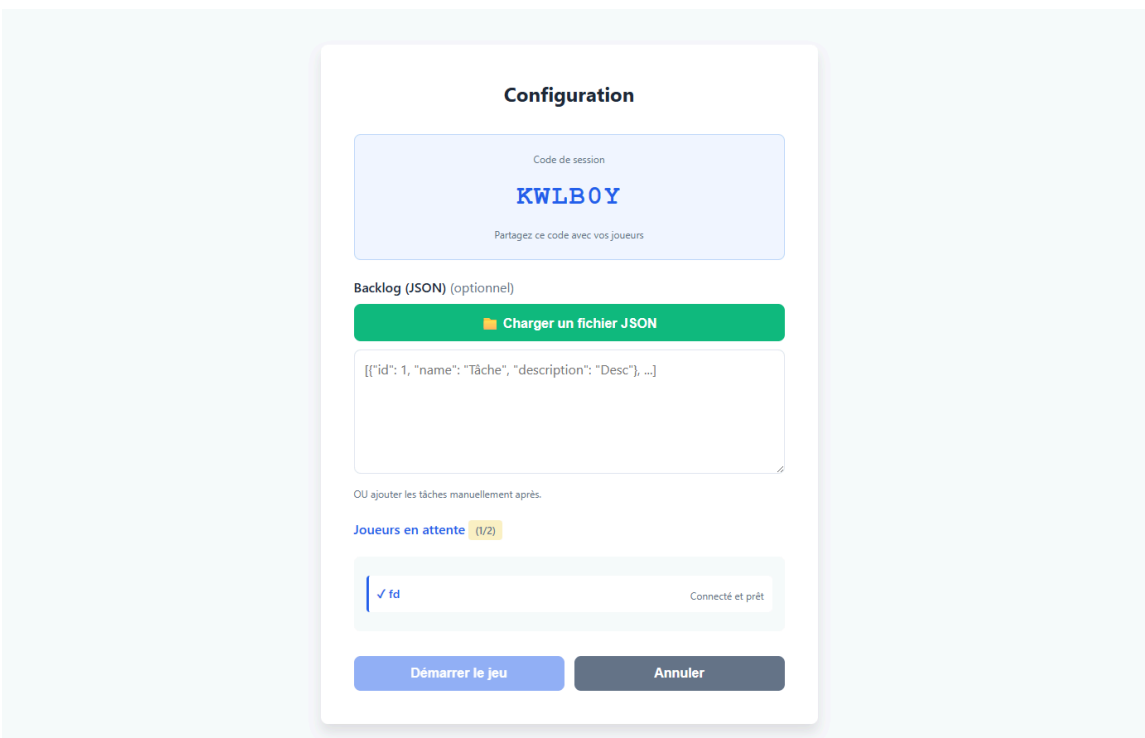
● Connecté



The screenshot shows the 'Planning Poker' main menu. It features a white card with a light blue border on a light blue background. The card has the title 'Planning Poker' at the top. Below the title, there are three input fields: 'Votre pseudo' with a placeholder 'Ex: Alice', 'Mode de jeu' with a dropdown menu showing 'Choisir un mode...', and 'Nombre de participants' with a dropdown menu showing 'Choisir le nombre...'. At the bottom of the card, there are three buttons: a blue button labeled 'Créer une partie', a grey button labeled 'Rejoindre', and a green button labeled 'Reprendre une partie' with a small icon.

### 1. Menu principal (`menuScreen`) :

- Permet de :
  - créer une nouvelle partie ;
  - rejoindre une partie existante ;
  - reprendre une partie à partir d'un fichier JSON de sauvegarde.



The screenshot shows the 'Configuration' screen. It features a white card with a light blue border on a light blue background. The card has the title 'Configuration' at the top. Below the title, there is a light blue box containing the text 'Code de session' and 'KWLBOY' in large blue letters, with a smaller text 'Partagez ce code avec vos joueurs' below it. Below this box, there is a green button labeled 'Charger un fichier JSON' with a small icon. Below the button, there is a text area containing a JSON string: '[{"id": 1, "name": "Tâche", "description": "Desc"}, ...]'. Below the text area, there is a small text 'OU ajouter les tâches manuellement après.' Below this, there is a section titled 'Joueurs en attente (1/2)' with a yellow background. Below this section, there is a light blue box containing a list of players: 'fd' with a checkmark icon and the text 'Connecté et prêt'. At the bottom of the card, there are two buttons: a blue button labeled 'Démarrer le jeu' and a grey button labeled 'Annuler'.

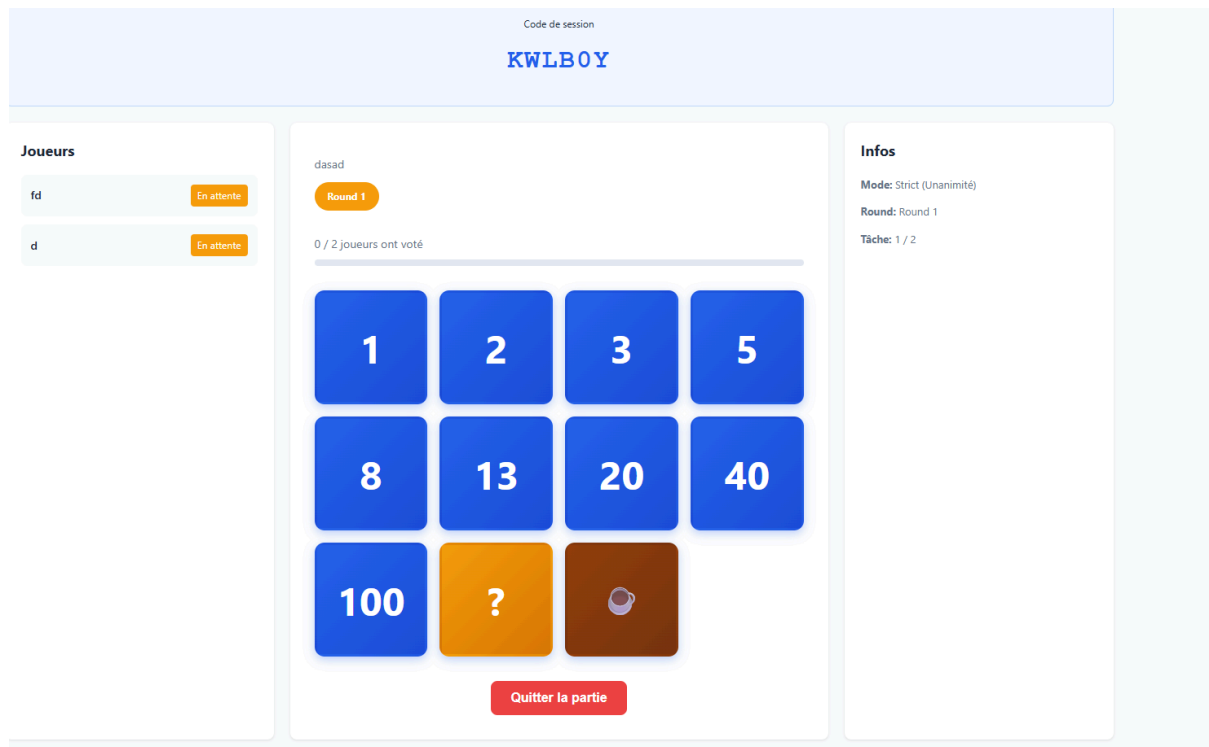
## 2. Création de session / configuration (`setupScreen`) :

- Le créateur saisit :
  - son pseudo ;
  - le mode de jeu ;
  - le nombre de participants attendus.
- Un code de session est généré et affiché pour être partagé aux autres joueurs.
- L'écran affiche :
  - la liste des joueurs connectés ;
  - un compteur (`X/Y`) ;
  - éventuellement l'interface d'ajout de tâches.

The screenshot shows a mobile application interface for adding tasks. At the top, there is a purple header with a plus icon and the text '+ Ajouter les tâches'. Below this, a light blue box displays the 'Code de session' (Session code) as 'KWLBOY'. Underneath, there are two input fields: 'Nom de la tâche' (Task name) with the example 'Ex: Ajouter authentification', and 'Description' with the example 'Ex: Implémenter login OAuth2'. A blue button with a plus icon and the text '+ Ajouter une tâche' is positioned below the description field. Further down, a section titled 'Tâches ajoutées (0)' (Added tasks (0)) shows a light green box with the text 'Aucune tâche pour le moment' (No task for the moment). At the bottom, there are two buttons: a blue button with a plus icon and the text '+ Commencer à voter' (Start voting), and a dark grey button with the text 'Retour' (Return).

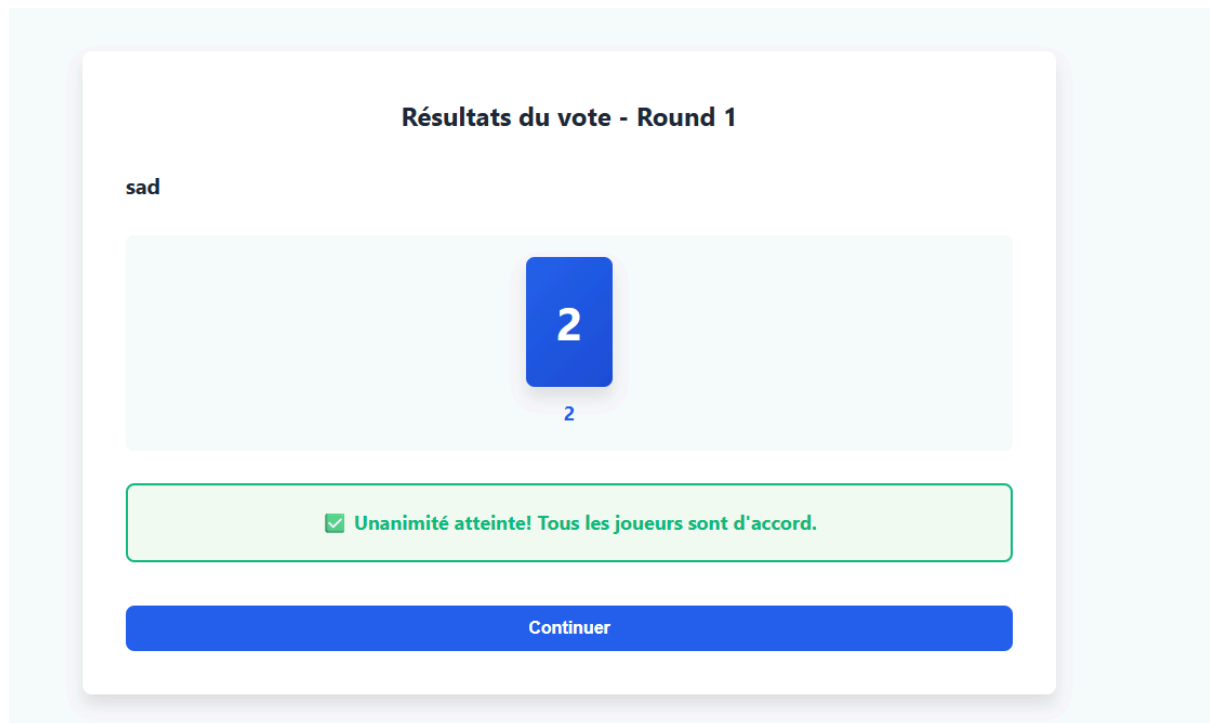
## 3. Ajout de tâches (`addTasksScreen`) :

- Le créateur peut :
  - ajouter des tâches manuellement (nom + description) dans un backlog local ;
  - charger un fichier JSON contenant un tableau de tâches ;
- Une fois le backlog prêt, il peut lancer le jeu.



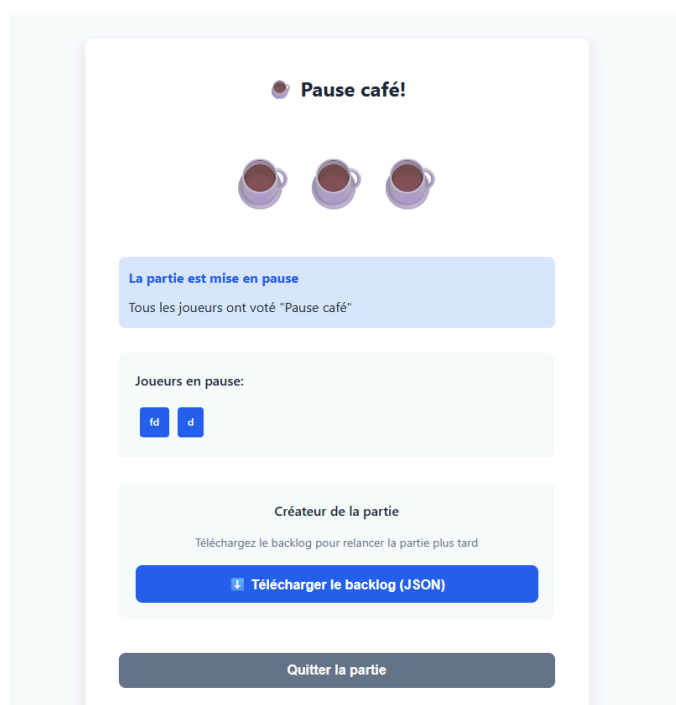
4. Écran de jeu (`gameScreen`) :

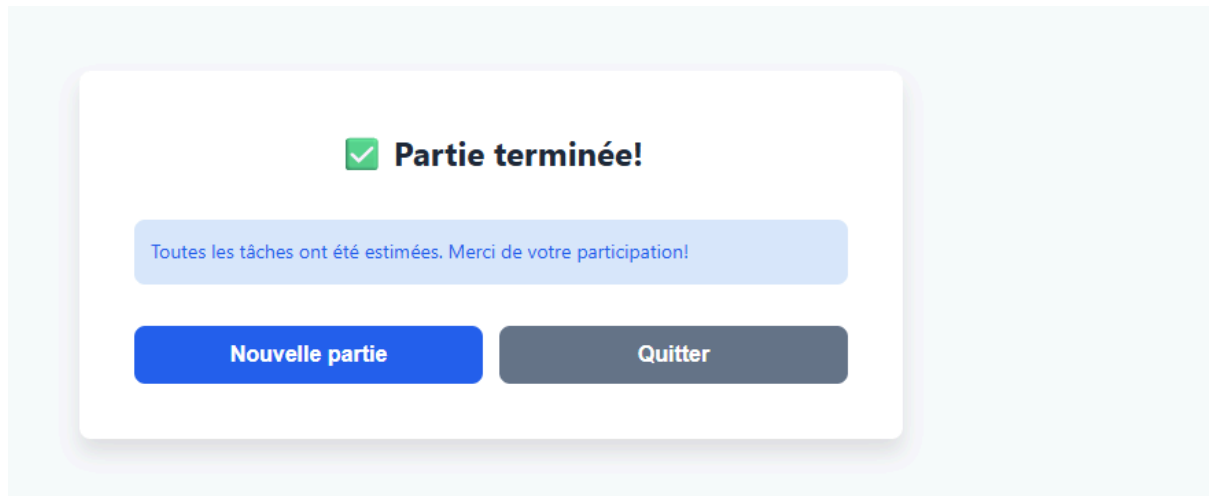
- Affiche la tâche courante (nom, description).
- Affiche la liste des joueurs et leur statut de vote via `PlayerList`.
- Affiche la barre de progression des votes avec `ProgressBar`.
- Affiche la grille de cartes `CardGrid` contenant les valeurs d'estimation (1, 2, 3, 5, 8, 13, 20, 40, 100, ? et ☕ pour une pause).
- Lorsque le joueur clique sur une carte, `castVote(cardValue)` est appelé et un événement `cast-vote` est envoyé au serveur.



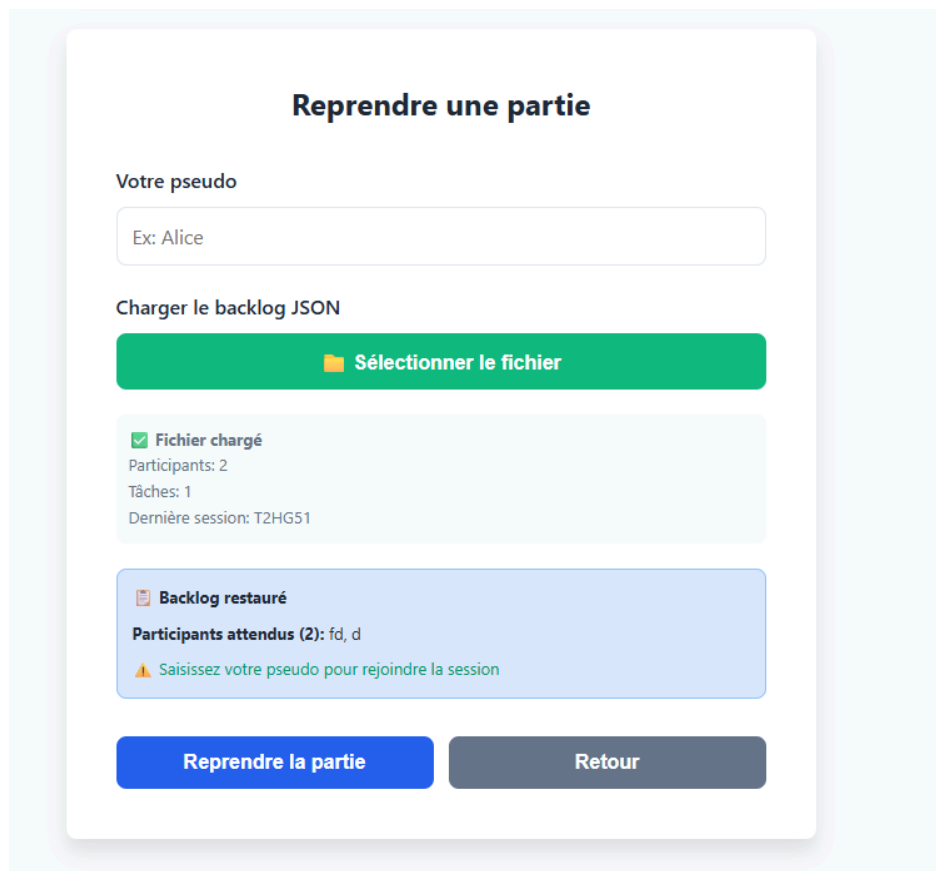
5. Résultats (`resultsScreen`, `taskCompletedScreen`) :

- Une fois tous les votes reçus (mode strict), les valeurs sont affichées à l'écran avec, selon la configuration, la valeur finale (moyenne/médiane).
- L'animateur peut :
  - valider la tâche et passer à la suivante (`continueToNext`) ;
  - relancer un round (`startNewRound`) si les votes sont trop dispersés ;
  - déclencher une pause café (écran dédié `coffeeBreakScreen`).





6. Fin de partie / sauvegarde (`gameOverScreen`) :
- À la fin du backlog, un écran de fin s'affiche.



7. **Reprise de partie (`resumeWaitingScreen`, `resumeSummaryScreen`) :**
- Lorsqu'un fichier de sauvegarde est importé, l'application reconstruit la session (participants attendus, backlog, votes existants).
- Les joueurs doivent se reconnecter avec leur pseudo pour être acceptés, conformément au fichier de sauvegarde.
- Cependant, cette fonctionnalité n'est pas entièrement fonctionnelle : après l'import de la sauvegarde, la partie ne se relance pas.**